HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

FINAL PROJECT REPORT

Topic: Parallelization Methods for CNN-Based Computer Vision Inference

Course: Parallel and Distributed Programming

Course code: IT4130E

Class/session: 168069 - Friday morning, periods 1–4, C7-215

Lecturer: Nguyen Tuan Dung

Group members: Luu Hieu An - 202400093

Nguyen Phu An - 20235466

Pham Chi Bang - 20235477

Nguyen Thanh Lam - 20235518

Hanoi, 2026

Contents

Abstract

Video people counting is an important computer vision problem for crowd monitoring, traffic analysis, safety management, and real-time camera systems, but applying a CNN detector to every frame is computationally expensive. The problem is also interesting from a parallel-programming perspective because video inference contains many repeated detector calls that can be decomposed across both time and space, while still requiring careful result merging to avoid duplicate counts near tile boundaries. This report presents *MPI-Based YOLO People Counting*, a C++17/OpenMPI pipeline that parallelizes pretrained YOLO inference on a three-MacBook physical cluster. The input video is first split temporally into frames and spatially into image tiles; each frame-tile pair becomes an MPI task assigned by static block-cyclic scheduling. Each rank runs a long-lived local YOLO worker on its assigned tasks, then rank 0 gathers detections, remaps tile-local coordinates to full-frame coordinates, removes duplicates with ownership filtering and global non-maximum suppression, computes frame-level people counts, and writes CSV timing artifacts. Dynamic master-worker scheduling is evaluated only as a comparison because its extra task-dispatch communication performs worse for this workload. The experiments separate parallel correctness from detector accuracy: the MPI implementation matches the serial pipeline on 30 tested frames with zero mismatches, while YOLO undercounts crowded MOT17-derived frames with a mean absolute count error of 7.983 and an average prediction of 8.223 people versus a ground-truth average of 16.207. For the required workload scale, $N = 600$ frames is selected because the 12-process run takes 123.667 seconds, and the $2N = 1200$-frame weighted 4/6/2 placement reaches 129.852 seconds with 2.662x wall-clock speedup. Overall, the results show that video people counting can be parallelized effectively as a task-level MPI workload, but performance depends strongly on communication overhead, tile granularity, and rank placement across heterogeneous machines.

Chapter 1

Introduction

Video people counting is a practical computer vision problem with applications in crowd monitoring, traffic analysis, safety management, and camera-based automation. A modern detector such as YOLO can identify people in individual images, but applying it repeatedly to every frame of a video quickly becomes computationally expensive. This makes the problem useful for a parallel-programming study: it is realistic enough to require a full inference pipeline, yet structured enough to expose clear opportunities for decomposition, scheduling, communication analysis, and speedup measurement.

This project focuses on inference rather than model training. The detector itself is a pretrained YOLO model, while the parallel-programming work is the C++17/OpenMPI runtime built around it. The system decomposes a video temporally into frames and spatially into image tiles, then treats each frame-tile pair as an MPI task. Each rank runs YOLO on its assigned tasks through a long-lived local worker process. After inference, rank 0 gathers detections, converts tile-local bounding boxes back to full-frame coordinates, removes duplicate detections near tile boundaries, computes frame-level people counts, and writes the experiment artifacts.

The report evaluates the implementation as a distributed-memory parallel program and separates parallel correctness from detector accuracy. Parallel correctness asks whether the MPI implementation preserves the output of the serial version of the same pipeline, while detector accuracy asks whether YOLO's predicted counts match human ground truth from MOT17-derived data. This distinction is important because the detector may undercount people in crowded scenes even when the MPI program is correct. The experiments therefore measure correctness, input-size suitability, scheduling behavior, granularity, speedup, and heterogeneous rank placement.

The main contributions and insights of this technical report are:

1. It formulates YOLO video people counting as a task-parallel MPI workload over frames and image tiles.
2. It presents an OpenMPI implementation that combines static block-cyclic task mapping, rank-local YOLO workers, centralized detection merging, duplicate removal, and metric collection.
3. It verifies that the MPI pipeline preserves serial output in the correctness experiment, while also showing that detector accuracy against ground truth is a separate limitation.
4. It compares scheduling and placement choices, showing that static scheduling and weighted rank placement are more suitable than dynamic dispatch for the measured heterogeneous cluster workload.
5. It analyzes speedup and remaining bottlenecks, especially communication overhead, tile granularity, master-side post-processing, and unequal machine capacity.

Chapter 2

Problem Description

2.1 CNN-Based Computer Vision Inference

The problem is to execute CNN-based video inference faster by distributing repeated detector work across MPI processes. The program applies a fixed pretrained model; it does not train or update model weights. The parallel system must preserve the output of the serial inference pipeline while reducing runtime or revealing the cost of communication.

2.2 YOLO Video People Counting

The input video is a finite sequence of frames:

$$V = \{F_1, F_2, \ldots, F_N\}. \tag{2.1}$$

The target output for each frame is the number of detected people after post-processing:

$$\mathit{count}(F_i) = |\{b \mid b \text{ is a final person bounding box in } F_i\}|. \tag{2.2}$$

The detector backend is a pretrained YOLO11n model through a local Python worker process [4]. YOLO11 follows the usual one-stage detector structure with a feature-extraction backbone, a feature-fusion neck, and multi-scale detection heads, as shown in Figure 2.1. The MPI program does not split YOLO neural-network kernels. Instead, MPI distributes repeated YOLO calls over frame-tile tasks.

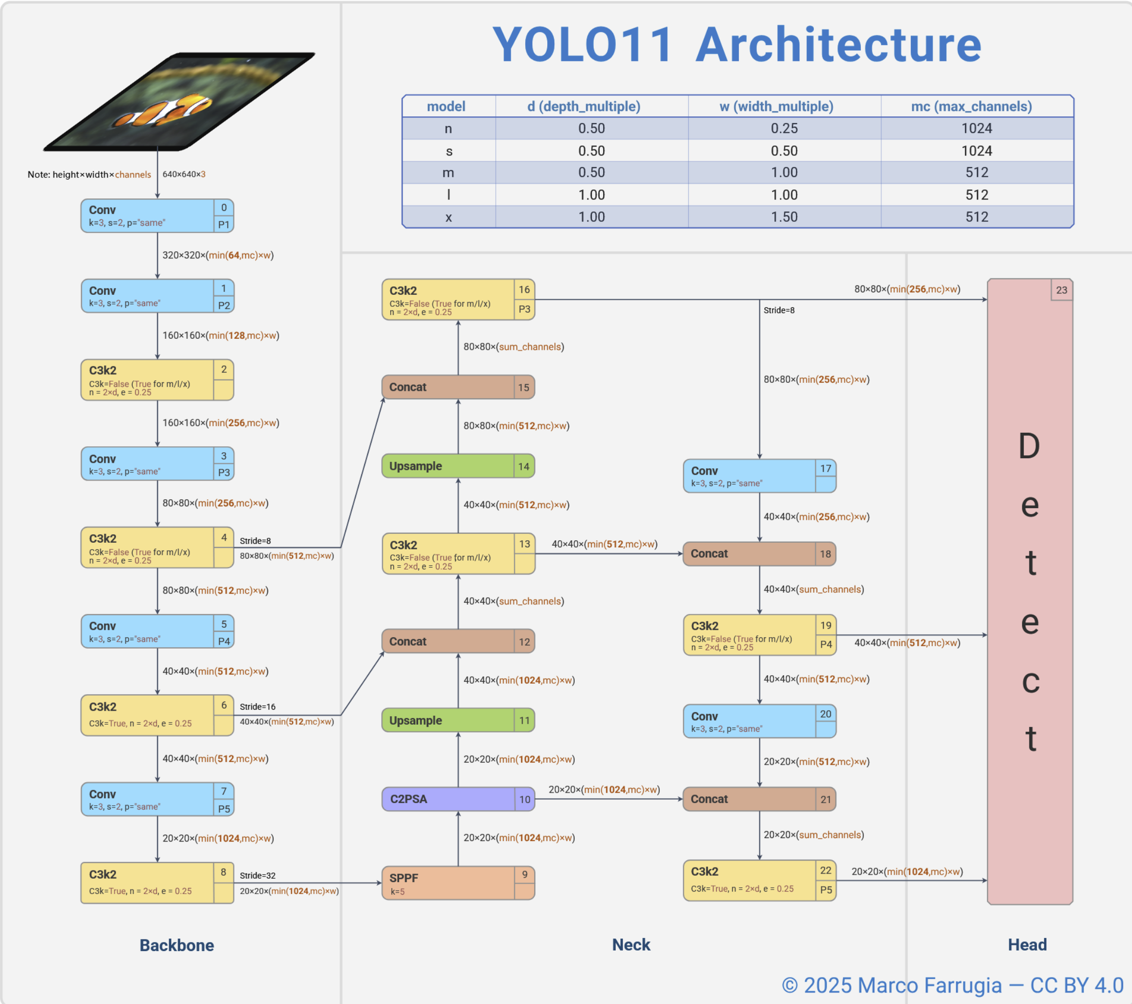


Figure 2.1: YOLO11 object-detection architecture used as the detector backend in this project. Source: Farrugia, CC BY 4.0, Zenodo DOI 10.5281/zenodo.17508018 [1].

For a tile grid with R rows and C columns, frame F_i is decomposed into:

$$F_i \to \{T_{i,1}, T_{i,2}, \ldots, T_{i,R\times C}\}. \tag{2.3}$$

A task is one frame-tile inference:

$$task(i,j) = T_{i,j}, \qquad |\mathcal{T}| = N \times R \times C. \tag{2.4}$$

Rank 0 converts tile-local detections back to full-frame coordinates, applies tile ownership filtering, removes duplicates, and writes final frame counts.

2.3 Parallel-Programming Focus

Table 2.1: Parallel-programming concepts applied in the YOLO MPI pipeline

Concept	Application in the YOLO MPI implementation
Parallelism level	Task-level parallelism. Each independent task corresponds to running YOLO inference on one frame tile.
Decomposition technique	Hybrid data decomposition: temporal decomposition over video frames combined with spatial decomposition over image tiles. The original video is decomposed into a one-dimensional list of frame-tile tasks.
Task granularity	Tile-level granularity. One task contains the inference workload for one tile of one frame. The tile size and number of tiles control whether the workload is fine-grained or coarse-grained.
Mapping technique	One-dimensional processor assignment using a flattened task list. Tasks are assigned to MPI ranks by static block-cyclic mapping. In the weighted version, faster devices receive a larger share of tasks.
Communication strategy	Communication is minimized during inference. Each rank performs local computation independently, then sends its local detections and timing metrics to rank 0 for global merging.
Communication topology	Master–worker topology. Worker ranks compute local YOLO detections, while rank 0 acts as the master process for gathering results, applying global post-processing, and writing output files.
Communication mode	Blocking collective communication is used in the current implementation during the gather phase. There is no communication between ranks during local YOLO inference.
Load balancing considerations	Load balancing is handled through tile-level task granularity, block-cyclic task distribution, and weighted process placement. The goal is to reduce idle time between MPI ranks, especially when machines have different computing speeds.
Correctness verification	The parallel result is considered correct if the final frame-level people counts match the serial YOLO pipeline after applying the same confidence filtering, coordinate remapping, and non-maximum suppression.
Performance evaluation	Execution time is measured with and without communication time. Speedup is evaluated by increasing the number of MPI processes while keeping the input size fixed at the selected benchmark size.

2.4 Correctness Targets

The serial YOLO baseline processes the same frame-tile task list without MPI distribution and applies the same post-processing pipeline. Parallel correctness is:

$$\Delta_i = |count_{parallel}(F_i) - count_{serial}(F_i)|. \tag{2.5}$$

The correctness experiment reports mismatched frames, maximum count error, and mean count error. Detector accuracy against MOT17 ground truth is evaluated separately because model error and parallelization error are different questions.

Chapter 3

Parallelization Methods

3.1 Task Parallel YOLO Inference

The YOLO pipeline uses task-level parallelism over independent frame-tile inference tasks. The implementation is centered on `src/yolo_mpi_cpp.cpp` in the repository. Each MPI rank runs the same C++ program, and rank-local detector work is delegated to a long-lived Python YOLO worker.

3.1.1 Temporal-Spatial Decomposition

The input video is first decomposed by time into frames and then by space into image tiles. If the video has N frames and each frame is split into an $R \times C$ grid, the total number of tasks is NRC. Figure 3.1 compares the original frame with 2x2 and 5x4 tiling. The 2x2 grid keeps detector accuracy close to the original-frame baseline because it introduces fewer artificial tile borders, so fewer people are cut by tile edges before the final merge stage. Larger grids create more tasks and can improve scheduling flexibility, but they increase duplicate detections near tile borders and increase post-processing work.

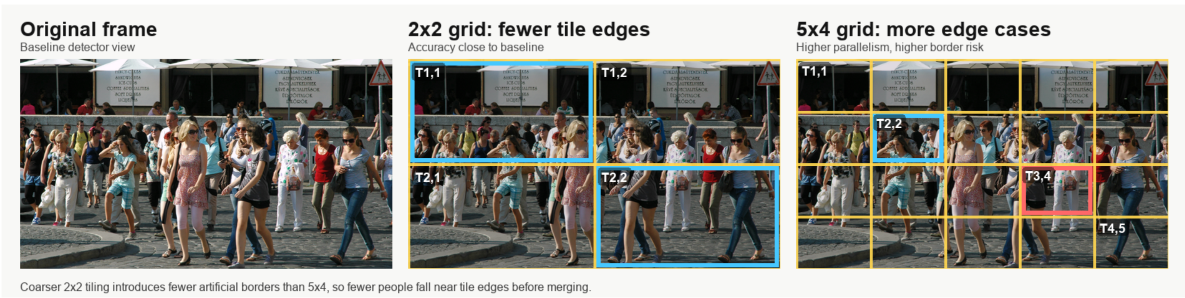


Figure 3.1: Hybrid temporal-spatial decomposition for YOLO video inference. The public-domain crowd frame is from Wikimedia Commons [2]; the overlays show why coarse 2x2 tiling preserves accuracy better than finer 5x4 tiling while still creating independent frame-tile tasks.

3.1.2 1D Mapping and Static Scheduling

All frame-tile tasks are flattened into a one-dimensional list. For task index i, chunk size k, and P ranks, static block-cyclic scheduling assigns:

$$owner(task_i) = \left\lfloor \frac{i}{k} \right\rfloor \bmod P. \tag{3.1}$$

Static scheduling has low dispatch overhead because ranks can determine their own tasks locally. The measured repository results show that this advantage matters for the 600-frame 5x4 tiled YOLO workload.

3.1.3 Communication and Merging

After each rank finishes its assigned frame-tile tasks, it sends two types of data to rank 0: detected bounding boxes and timing metrics. The communication pattern is centralized. Worker ranks compute locally, then rank 0 gathers the local result payloads and performs the final merge.

For each detected person box, the worker stores:

$$(frame\_id,\ tile\_id,\ x_1,\ y_1,\ x_2,\ y_2,\ score,\ class). \tag{3.2}$$

The box coordinates are still relative to the local tile. Rank 0 first converts them back to original-frame coordinates:

$$x^{global} = x^{local} + x_s, \qquad y^{global} = y^{local} + y_s, \tag{3.3}$$

where (x_s, y_s) is the top-left coordinate of the tile inside the original frame.

For each frame F_i, rank 0 merges all detections from the $R \times C$ tiles:

$$B_i^{raw} = \bigcup_{j=1}^{RC} B_{i,j}. \tag{3.4}$$

Since a person near a tile boundary may appear in more than one tile, duplicate boxes can occur. Rank 0 therefore applies tile-owner filtering followed by global non-maximum suppression:

$$B_i^{final} = NMS(B_i^{raw}, \theta). \tag{3.5}$$

The final people count for frame F_i is:

$$count(F_i) = |B_i^{final}|. \tag{3.6}$$

This centralized merge is necessary because each rank only sees its own assigned tiles, while duplicate removal requires all detections from the same frame.

3.1.4 YOLO Algorithm Sketch

Algorithm 1 Static MPI YOLO people-counting pipeline

Input video contains N frames; each frame is split into an $R \times C$ grid.
Build task list $\mathcal{T}$, where each task is $(frame\_id, tile\_id)$.
Flatten $\mathcal{T}$ into a 1D list and assign task index q using

$$owner(q) = \left\lfloor \frac{q}{K} \right\rfloor \bmod P,$$

where K is chunk size and P is the number of MPI ranks.
for each MPI rank r in parallel **do**
  Select all tasks whose owner is r.
  for each selected task (i, j) **do**
    Crop tile j from frame F_i.
    Run YOLO inference on the tile.
    Keep only `person` detections.
    Store local boxes and timing metrics.
  end for
end for
Gather serialized detections and rank metrics to rank 0.
if rank is 0 **then**
  for each frame F_i **do**
    Convert tile-local boxes to global frame coordinates.
    Merge boxes from all tiles of frame F_i.
    Apply tile-owner filtering and global NMS.
    Compute $count(F_i) = |B_i^{final}|$.
  end for
  Write per-frame counts and runtime metrics to CSV files.
end if

3.2 Parallel-Programming Focus

Table 3.1: Parallel-programming concepts in the YOLO MPI pipeline

Concept	YOLO MPI implementation
Parallelism level	Task-level parallelism: each task runs YOLO on one frame tile.
Decomposition	Hybrid temporal-spatial decomposition: video frames are divided into smaller image tiles.
Granularity	Tile-level granularity: one task corresponds to one tile of one frame.
Mapping technique	1D task mapping: all frame-tile tasks are flattened into a list and assigned to MPI ranks using static block-cyclic distribution.
Communication strategy	Each rank computes locally, then sends detections and timing results to rank 0 for merging.
Communication topology	Master-worker topology: rank 0 gathers and post-processes results, while other ranks mainly perform inference.
Communication mode	Blocking gather communication is used after local inference.
Load balancing	Load balancing is considered through tile size, block-cyclic task assignment, and weighted process placement.
Correctness check	The parallel output is correct if its frame-level people counts match the serial YOLO result after the same post-processing.

Chapter 4

Experimental Setup

4.1 Repository Evidence

The report is based on the local repository `yolo-mpi-people-count`. The strongest source for measured YOLO values is `reports/final_report_hust_style.md`, with additional full-cluster discussion in `reports/additional_experiments_20260623.md`. The artifact branch also provides compact CSV files under `reports/artifacts/`; the YOLO MPI CSV artifacts have been copied into this report repository under `artifacts/`.

4.2 Physical Cluster

The project uses three physical MacBook machines connected through a local network. The benchmark mode uses CPU execution to match the course emphasis on MPI processes and processor assignment. GPU/MPS execution is kept for demonstration and live-camera use rather than formal CPU speedup measurement.

Table 4.1: Physical cluster roles

Host	Machine type	Role in experiments
master	MacBook Air M4	Coordinates execution, gathers results, stores outputs, and can run local tasks.
node1	MacBook Pro M4	Worker host; measured results suggest it can sustain more assigned ranks.
node2	MacBook Air M2	Worker host; weighted placement assigns fewer ranks to reduce imbalance.

4.3 Software and Data

The implementation uses C++17, OpenMPI, Python helper scripts, and a pretrained Ultralytics YOLO detector. The main dataset source is MOT17-derived video data [3]. Compact MOT17 subsets are used for quick correctness and granularity experiments; longer sequence segments are used for the required input-size and speedup runs.

Table 4.2: Main YOLO experimental configuration

Item	Setting
Detector	Pretrained YOLO11n, person-class filtering
Parallel framework	C++17 and OpenMPI
Benchmark device	CPU
Dataset	MOT17-derived sequences
Main process count	12 MPI processes
Main long-run grid	5x4 regions
Scheduler	Static block-cyclic task assignment
Placement variants	Uniform 4/4/4 and weighted 3/6/3, 4/6/2

4.4 Metrics

The report uses the following metrics from the project reports and generated CSV design:

- serial-versus-MPI correctness: mismatched frames and count error;
- detector accuracy: MAE, RMSE, percentage error, exact-match ratio;
- runtime with communication: wall-clock time including coordination;
- runtime without communication: compute-oriented time reported by the scripts;
- speedup and efficiency:

$$S_p = \frac{T_1}{T_p}, \qquad E_p = \frac{S_p}{p}; \tag{4.1}$$

- idle-gap indicator for load-balance analysis.

Chapter 5

Results and Discussion

5.1 Detector Accuracy Under Tile Splitting

Table 5.1 compares detector accuracy against MOT17 ground-truth counts for the normal single-machine YOLO baseline and for different MPI tile-splitting configurations. The purpose of this experiment is to check whether spatial tiling changes the final detection accuracy, not only whether the MPI implementation is faster.

Table 5.1: YOLO accuracy comparison across baseline and tile-splitting configurations

Configuration	Grid	Frames	Runtime (s)	MAE	RMSE	Percent err.	Pred. avg.
Single-machine baseline	1x1	300	–	7.983	8.269	0.490	8.223
MPI tiled	2x2	300	–	7.983	8.269	0.490	8.223
MPI tiled	4x3	300	–	8.223	8.517	0.504	7.977
MPI tiled	5x4	300	54.764	8.383	8.682	0.514	7.812

The runtime entry is included only where the repository contains an official measurement for the same 300-frame workload. The 2x2 configuration is effectively equivalent to the single-machine baseline, so moderate spatial tiling does not change detector behavior. Finer grids introduce a small degradation: the 4x3 configuration is about 3 percent worse than baseline, and the 5x4 configuration is about 5 percent worse. This suggests that aggressive tile splitting can harm accuracy through border effects and imperfect cross-tile merging, even when it creates more parallel work.

5.2 Input-Size Selection

The compact MOT17 subset is useful for quick runs but does not reach the required two-to-three-minute runtime. Table 5.2 also reports the total number of frame-tile tasks and the wall-clock time per task, making it easier to compare efficiency across input sizes. The long-sequence experiment selects $N = 600$ frames because the 12-process wall-clock runtime is 123.667 seconds.

Table 5.2: YOLO input-size selection

Frames	P	Grid	Tasks	Runtime with comm. (s)	Runtime without comm. (s)	ms/task	Selection note
30	12	4x3	360	9.165	2.999	25.46	Compact quick test
60	12	4x3	720	12.570	6.801	17.46	Compact quick test
100	12	4x3	1200	17.776	11.596	14.81	Compact quick test
150	12	4x3	1800	23.585	17.103	13.10	Compact quick test
300	12	5x4	6000	54.764	49.156	9.13	Long sequence
600	12	5x4	12000	123.667	114.352	8.97	Selected N
837	12	5x4	16740	146.316	139.136	8.74	Best ms/task, longer validation
1200	12	5x4	24000	129.852	128.122	5.41	$2N$, weighted run

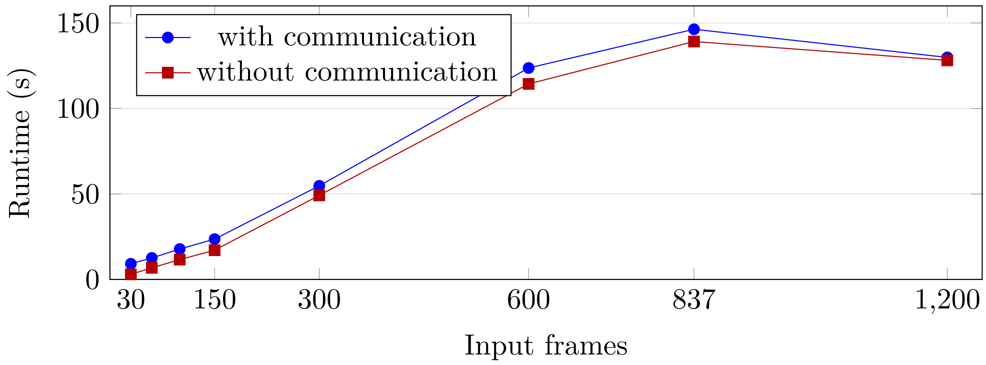


Figure 5.1: Runtime versus input size. The 600-frame workload is the first measured case inside the required 120–180 second interval.

The time-per-task column shows that very small inputs have high overhead because startup, synchronization, and aggregation are amortized over too few tasks. The 600-frame run is selected as N because it is the first long-sequence case that satisfies the required 120–180 second runtime window. The 1200-frame row is the $2N$ speedup workload.

5.3 Granularity and Load Balance

Table 5.3 summarizes the official granularity experiment using weighted 6/4/2 placement. The weighted placement keeps the idle-gap indicator at 0.235, within the 25 percent course criterion. Increasing the number of regions still increases task count and communication, so granularity remains a performance trade-off even when load balance is acceptable.

Table 5.3: YOLO granularity and load balance with weighted 6/4/2 placement

Grid	P	Tasks	Max comp. (s)	Avg comp. (s)	Total comm. (s)	Idle gap	Balance
1x1	12	150	0.731	0.483	10.674	0.235	Balanced
2x2	12	600	3.155	2.205	16.306	0.235	Balanced
4x3	12	1800	9.266	6.375	34.881	0.235	Balanced
5x4	12	3000	13.733	9.133	51.790	0.235	Balanced

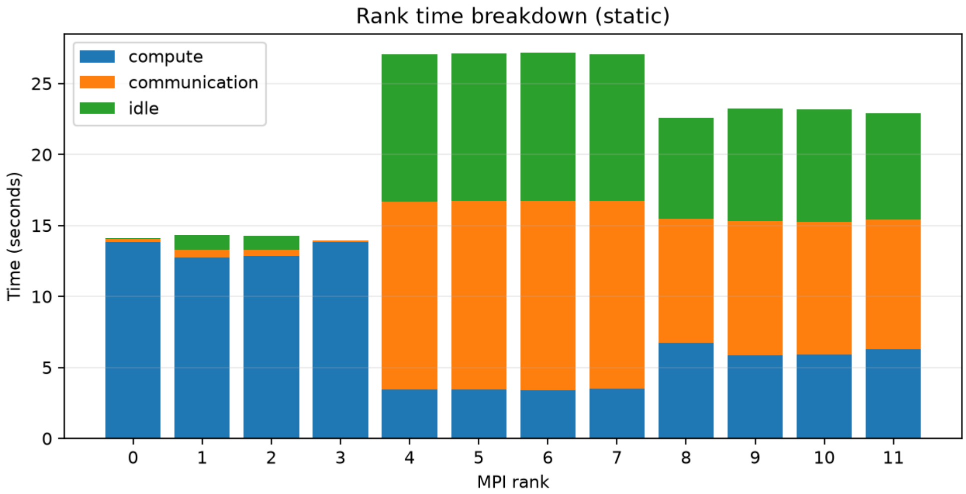


Figure 5.2: Per-process timing breakdown for the granularity experiment. Each bar is one MPI process, with compute, communication, and idle time stacked in the same column.

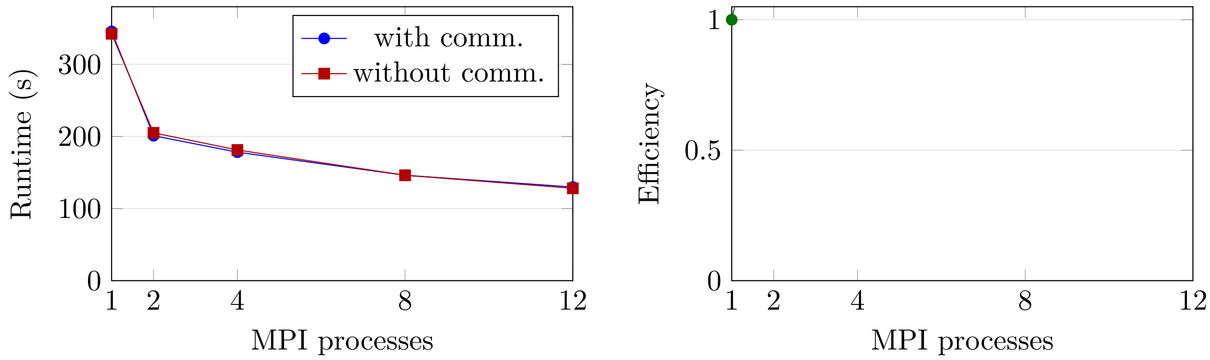
The result shows that weighted placement can satisfy the idle-gap criterion, but simply making tasks finer is not enough to guarantee faster execution. Finer tiles create more scheduling opportunities, but they also increase intermediate detections, communication, and master-side merging. Figure 5.2 shows why the load-balance check must inspect process-level timing rather than only total runtime.

5.4 Speedup and Efficiency

The speedup experiment uses the selected input size doubled to $2N = 1200$ frames and applies weighted static placement for the multi-rank runs. This keeps the process distribution consistent with the heterogeneous cluster: stronger machines receive more ranks, while the slower machine receives fewer ranks.

Table 5.4: YOLO speedup on 1200 frames with weighted placement

P	Runtime with comm. (s)	Runtime without comm. (s)	Wall speedup	Efficiency	Note
1	345.677	342.467	1.000	1.000	Baseline
2	201.111	205.100	1.719	0.859	Weighted placement
4	178.238	181.289	1.939	0.485	Weighted placement
8	146.212	146.204	2.364	0.296	Weighted placement
12	129.852	128.122	2.662	0.222	Weighted 4/6/2



(a) Runtime by process count (b) Wall-clock speedup

Figure 5.3: Runtime and speedup trends for the 1200-frame weighted-placement run.

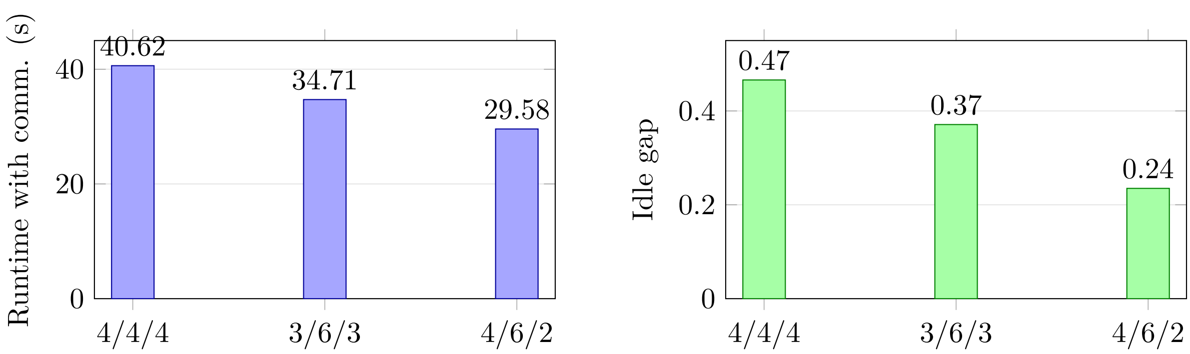
The speedup is measurable but not linear. Under weighted placement, the wall-clock speedup rises from 1.719x at two processes to 2.662x at twelve processes. The 12-process computation-only speedup is approximately 2.673x. The improvement comes from reducing idle gap and assigning fewer ranks to the slower node.

5.5 Weighted Static Placement

The equal 4/4/4 placement does not meet the 25 percent idle-gap criterion. The best measured weighted placement assigns four ranks to the master, six to node1, and two to node2.

Table 5.5: Weighted static placement on heterogeneous cluster

Placement	Distribution	Runtime with comm. (s)	Runtime without comm. (s)	Idle gap	Balance
Uniform	4/4/4	40.619	36.522	0.466	Not balanced
Weighted	3/6/3	34.712	30.339	0.371	Not balanced
Weighted	4/6/2	29.581	27.484	0.235	Balanced



(a) Runtime by placement (b) Idle gap by placement

Figure 5.4: Weighted placement reduces both runtime and idle-gap imbalance.

This result supports the claim that processor assignment matters on a heterogeneous physical cluster. Equal rank placement gives every host the same number of processes, but the machines do not have equal throughput. Weighted placement reduces runtime and satisfies the load-balance criterion.

5.6 YOLO Parallel Correctness

Table 5.6 reports the serial-versus-MPI correctness result recorded in the repository report. The MPI run produced the same frame counts as the serial baseline on the tested sequence.

Table 5.6: YOLO serial-versus-MPI correctness

Result	Frames	Mismatched frames	Max count error	Mean error	Evidence
Passed	30	0	0	0.000	repository report

This validates the parallel pipeline for the tested configuration. It indicates that task splitting, MPI result collection, coordinate remapping, duplicate removal, and final counting preserve the serial pipeline output.

5.7 Summary of Findings

The measured YOLO results support the main task-parallel narrative. The implementation preserves serial output, reaches the required input-size runtime, shows measurable speedup, and demonstrates that task granularity and rank placement strongly affect performance. The strongest measured improvement in load balance comes from weighted static placement, which reduces the idle-gap indicator from 0.466 to 0.235.

Chapter 6

Conclusion

This report presents the final content for *Parallelization Methods for CNN-Based Computer Vision Inference* using evidence from the `yolo-mpi-people-count` repository. The completed YOLO method demonstrates task-level parallelism with hybrid temporal-spatial decomposition, 1D task mapping, static block-cyclic scheduling, rank-local detector execution, and centralized detection merging.

The measured YOLO results are strong enough for the main experimental narrative. The serial-versus-MPI correctness test passes with zero mismatched frames. The selected 600-frame workload runs in 123.667 seconds with 12 processes, satisfying the required two-to-three-minute range. The official 1200-frame weighted 4/6/2 run reaches 129.852 seconds and 2.662x wall-clock speedup. Weighted static placement improves the idle-gap indicator to 0.235 and satisfies the 25 percent load-balance criterion.

The detector accuracy result should be interpreted separately from parallel correctness. YOLO undercounts crowded MOT17 frames, with MAE 7.983 on the recorded 300-frame accuracy experiment. This is a model limitation, not evidence that MPI changed the pipeline output.

Overall, the project covers the core course concepts through one coherent method: decomposition, mapping, communication topology, scheduling, load balancing, correctness verification, input-size selection, and speedup measurement.

Bibliography

[1] Marco Farrugia. Yolo11 architecture diagram: Visual representation of the yolo11 object detection model. https://doi.org/10.5281/zenodo.17508018, 2025.

[2] Keymeulen. Crowd crossing the street, hungary 2011. https://commons.wikimedia.org/wiki/File:Crowd_crossing_the_street,_Hungary_2011.jpg, 2011. CC0 1.0 Universal Public Domain Dedication.

[3] MOTChallenge. Mot17 benchmark. https://motchallenge.net/data/MOT17/, 2026.

[4] Ultralytics. Ultralytics yolo documentation. https://docs.ultralytics.com/, 2026.